

# SOFTWARE DESIGN DESCRIPTION (SDD)

Doctor Appointment System Web Application

---

**Project Title:** Doctor Appointment System

**Document Version:** 1.0

**Date:** June 7, 2026

**Author:** Ahmed Tehseen

**Roll Number:** S25BINFT7M04061

**Department:** Information Technology

Project Supervisor : Mam

Sara Farid

.

## Document Control Information

| Version | Date         | Author / Modifier | Description                                   |
|---------|--------------|-------------------|-----------------------------------------------|
| 1.0     | June 7, 2026 | Ahmed Tehseen     | Initial Architecture and Design Specification |

## 1. Introduction

### 1.1 Purpose

This Software Design Description (SDD) document details the internal architecture, system modules, interfaces, and data models for the Doctor Appointment System. It translates the requirements outlined in the Software Requirements Specification (SRS) into a concrete, implementable engineering design framework for web development.

### 1.2 Scope

The Doctor Appointment System is a web-based portal providing an interface for patients to search, select, and schedule clinical appointments, while giving healthcare professionals and system administrators tools to manage calendars, track schedules, view patient histories, and regulate access control matrices. This document describes the component-level view, interface configurations, and data structure designs required for engineering the platform.

### 1.3 Acronyms and Abbreviations

- **SDD:** Software Design Description
- **MVC:** Model-View-Controller
- **REST:** Representational State Transfer
- **API:** Application Programming Interface
- **UI/UX:** User Interface / User Experience
- **RDBMS:** Relational Database Management System
- **JWT:** JSON Web Token

## 2. System Architecture and High-Level Design

### 2.1 Architectural Pattern (MVC)

The application implements an architectural separation via the Model-View-Controller (MVC) and Client-Server framework patterns to decouple storage mechanics, logical data manipulation, and view presentations.

- **Client / View Layer:** Constructed via semantic HTML5, CSS3 structural layout sheets, and interactive JavaScript UI components handling client-side inputs and view updates.
- **Server / Controller Layer:** Exposes standardized secure RESTful API endpoints handling state management, operations authorization, routing, and database interaction.
- **Storage / Model Layer:** Relational database infrastructure running transaction-safe management operations representing concrete entities, constraints, and relational entities.

### 2.2 High-Level Architecture Diagram

The following diagram depicts the operational mapping, communication interfaces, and runtime relationship between system tiers:

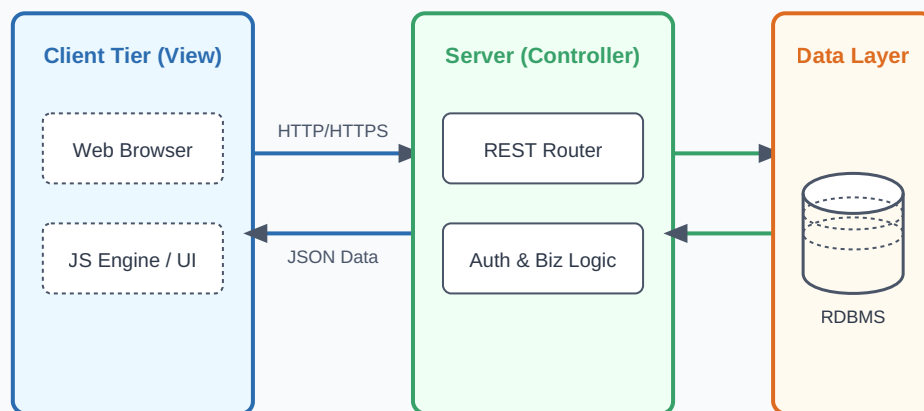


Figure 2.1: 3-Tier High-Level Architectural Diagram

## 3. System Modules and Interface Architecture

The platform separates operations contextually into three explicit user roles, backed by cross-cutting platform automation modules:

### 3.1 Patient Module

- **Profile Initialization:** Creates and maintains standard patient demographical, biological, and medical indexing records securely.
- **Medical Directory Browser:** Allows advanced filtering of active medical specialists by classification, regional availability, charging fees, or open dates.
- **Booking Interface:** Coordinates validation constraints preventing appointment timing conflicts and double bookings.

### 3.2 Doctor Management Module

- **Calendar Management Matrix:** Enables clinicians to configure active profile slot states, break parameters, and active clinical times.
- **Patient Record Access Layer:** Authorizes view privileges for upcoming patients' consultation timelines and history files.

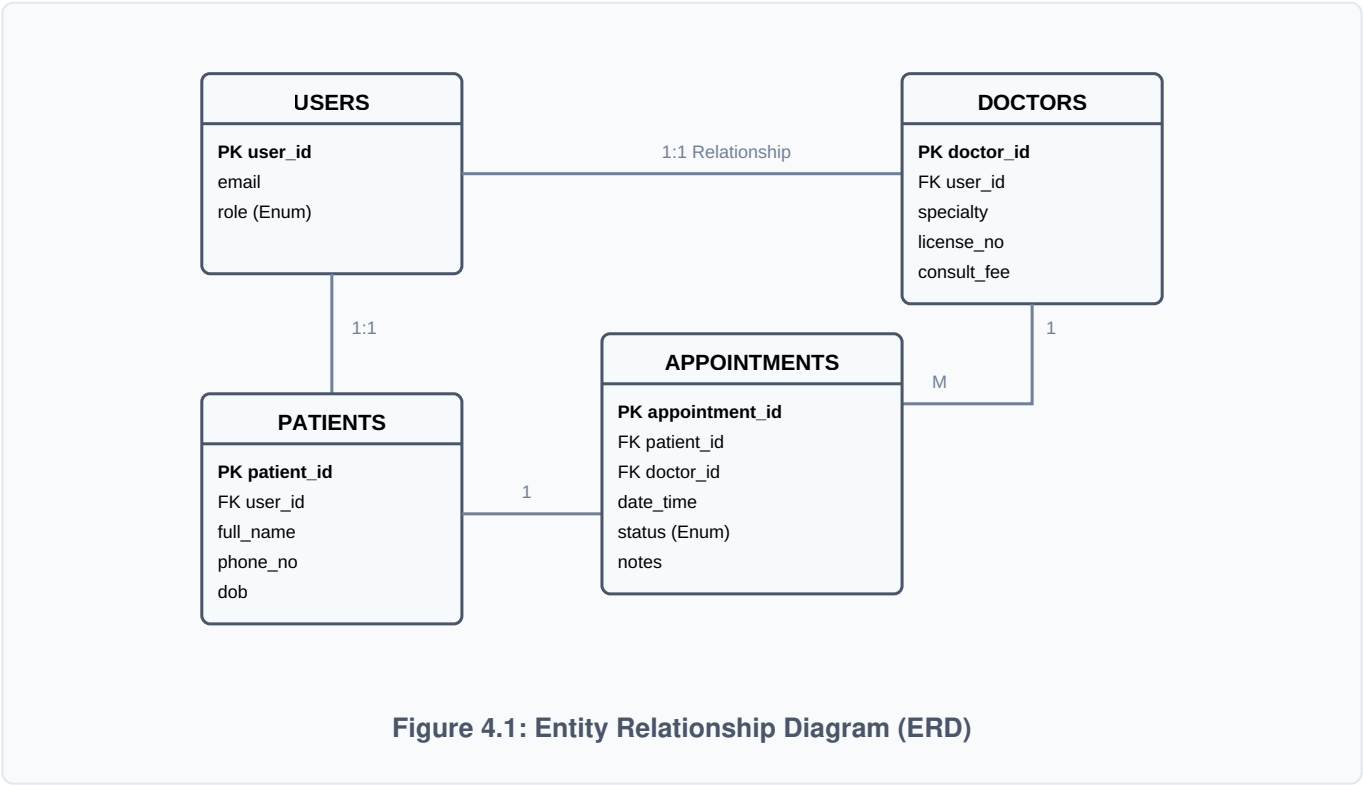
### 3.3 Administrator Control Engine

- **Master Governance Dashboard:** System telemetry monitor to review real-time booking statistics, manage system security profiles, and generate reports.

## 4. Data Model and Database Design

The system stores operational data inside an entity-relational database. Relational schemas map directly to platform models below.

4.1 Entity Relationship Diagram (ERD)



4.2 Data Dictionary Details

Table 4.2.1: APPOINTMENTS Realization Schema

| Column Element | Data Type      | Constraint Rules                 | Functional Description Description                                         |
|----------------|----------------|----------------------------------|----------------------------------------------------------------------------|
| appointment_id | BIGINT / UUID  | PRIMARY KEY, AUTO_INCREMENT      | Unique surrogate signature distinguishing booking instances.               |
| patient_id     | BIGINT         | FOREIGN KEY referencing PATIENTS | Identifies the patient associated with the booking.                        |
| doctor_id      | BIGINT         | FOREIGN KEY referencing DOCTORS  | Identifies the doctor associated with the booking.                         |
| date_time      | DATETIME       | NOT NULL                         | The scheduled date and start time for the appointment.                     |
| status         | VARCHAR / ENUM | DEFAULT 'PENDING'                | State control trackers: ['PENDING', 'CONFIRMED', 'COMPLETED', 'CANCELLED'] |

## 5. Interface and System Behavior Design

This section outlines behavioral processing rules and execution flow sequences during platform state modifications.

### 5.1 Booking Sequence Mechanics

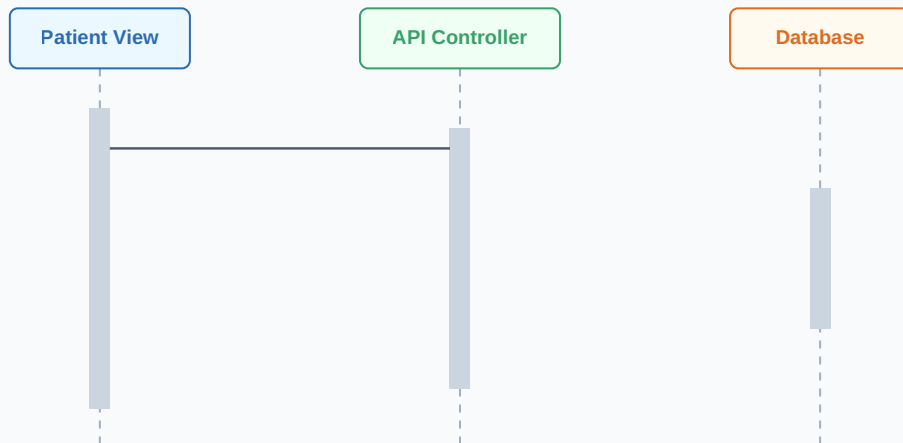


Figure 5.1: Sequence Diagram of the Appointment Booking Flow

## 6. Security and Quality Control Specifications

### 6.1 Authentication Architecture

User interactions across protected web interfaces require authorization validation headers using encrypted JSON Web Tokens (JWT). The transmission protocol specifies that all sensitive communication passes exclusively over Transport Layer Security (TLS 1.3 / HTTPS).

### 6.2 Input Sanitization & Data Validation

To mitigate security vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS), all inputs undergo multi-stage sanitization rules before execution, as demonstrated in the architectural controller logic sample below:

```
// Controller Security Pattern: Request Validation & Sanitization
function handleBookingRequest(req, res) {
  const sanitizedPatientId = sanitizeInput(req.body.patientId);
  const sanitizedDoctorId = sanitizeInput(req.body.doctorId);
  const appointmentDateTime = parseIsoDatetime(req.body.dateTime);
```

```
    if (!validateSlotAvailability(sanitizedDoctorId, appointmentDateTime)) {
        return res.status(409).json({ error: "Requested time slot conflict
detected." });
    }

    db.execute("INSERT INTO appointments (patient_id, doctor_id, date_time, status)
VALUES (?, ?, ?, 'PENDING'",
        [sanitizedPatientId, sanitizedDoctorId, appointmentDateTime]);

    return res.status(201).json({ message: "Appointment request registered
successfully." });
}
```